PRAKTIKUM 5

# POSTGRESQL, PGADMIN, VOLUME, BACKUP, DAN RESTORE

Disusun untuk Memenuhi Matakuliah Workshop DevOps

Dosen Pengampu : Bapak Dr. Ferry Astika Saputra, S.T., M.Sc.




Oleh :

Nama : Putra Yanuar Rasyid

NRP : 3126640058

Politeknik Elektronika Negeri Surbaya

Departemen Teknik Informatika dan Komputer

Program Studi Teknik Informatika


2026

**Praktikum 5 - Menetapkan Baseline Laboratorium**

Inisialisasi struktur direktori proyek lab

```
ras1putra@ras1putra:~$ mkdir -p ~/docker-lab/bab-5/{init,backup,pgadmin,scripts}
ras1putra@ras1putra:~$ cd ~/docker-lab/bab-5
```

Analisis: Membuat direktori kerja Bab 5 beserta 4 sub-direktori isolasi untuk skrip inisialisasi DDL (init), pencadangan (backup), konfigurasi pgAdmin (pgadmin), dan otomasi skrip (scripts)

Pembuatan file environment dan pengamanan hak akses

```
ras1putra@ras1putra:~/docker-lab/bab-5$ cat << 'EOF' > .env.example
POSTGRES_DB=labdb
POSTGRES_USER=labuser
POSTGRES_PASSWORD=labpass123
PGADMIN_DEFAULT_EMAIL=admin@example.local
PGADMIN_DEFAULT_PASSWORD=admin123
EOF
ras1putra@ras1putra:~/docker-lab/bab-5$ cp .env.example .env
ras1putra@ras1putra:~/docker-lab/bab-5$ chmod 600 .env
ras1putra@ras1putra:~/docker-lab/bab-5$
```

Analisis: Mengisolasi kredensial database dan pgAdmin ke dalam file .env serta mengunci hak akses berkas menjadi chmod 600 guna mencegah akses non-authorized pada host

Konfigurasi proteksi repositori git (.gitignore)

```
ras1putra@ras1putra:~/docker-lab/bab-5$ cat << 'EOF' > .gitignore
.env
backup/*.dump
backup/*.sha256
backup/*.log
EOF
ras1putra@ras1putra:~/docker-lab/bab-5$
```

Analisis: Mendaftarkan file .env dan direktori dump backup ke dalam .gitignore untuk mencegah kebocoran kredensial dan arsip database ke repositori Git publik

Konfigurasi multi-container stack compose (compose.yaml)

```
ras1putra@ras1putra:~/docker-lab/bab-5$ cat << 'EOF' > compose.yaml
services:
  postgres-db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports:
      - "127.0.0.1:5432:5432"
    volumes:
      - pg-data:/var/lib/postgresql/data
      - ./init:/docker-entrypoint-initdb.d:ro
    networks:
      - data-net
    healthcheck:
      test:
        - CMD-SHELL
        - "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"
      interval: 5s
      timeout: 5s
      retries: 10
      start_period: 10s
    restart: unless-stopped

  pgadmin:
    image: dpage/pgadmin4:latest
    environment:
      PGADMIN_DEFAULT_EMAIL: ${PGADMIN_DEFAULT_EMAIL}
      PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_DEFAULT_PASSWORD}
    ports:
      - "127.0.0.1:5050:80"
    volumes:
      - pgadmin-data:/var/lib/pgadmin
      - ./pgadmin/servers.json:/pgadmin4/servers.json:ro
    networks:
      - data-net
    depends_on:
      postgres-db:
        condition: service_healthy
    restart: unless-stopped

volumes:
  pg-data:
  pgadmin-data:

networks:
  data-net:
    driver: bridge
EOF
ras1putra@ras1putra:~/docker-lab/bab-5$
```

Analisis: Mendefinisikan service postgres-db (PostgreSQL 16) dengan volume persisten pg-data, probe pg_isready, network privat data-net, serta service GUI pgadmin.

Skrip inisialisasi sql dan konfigurasi server pgadmin

```
ras1putra@ras1putra:~/docker-lab/bab-5$ cat << 'EOF' > init/01-schema.sql
CREATE TABLE IF NOT EXISTS students (
    id BIGSERIAL PRIMARY KEY,
    nrp VARCHAR(20) UNIQUE NOT NULL,
    name TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO students (nrp, name)
VALUES
    ('31230001', 'Mahasiswa Satu'),
    ('31230002', 'Mahasiswa Dua')
ON CONFLICT (nrp) DO NOTHING;

CREATE INDEX IF NOT EXISTS idx_students_name
    ON students (name);
EOF
ras1putra@ras1putra:~/docker-lab/bab-5$ cat << 'EOF' > pgadmin/servers.json
{
  "Servers": {
    "1": {
      "Name": "PostgreSQL Bab 5",
      "Group": "Laboratorium DevSecOps",
      "Host": "postgres-db",
      "Port": 5432,
      "MaintenanceDB": "labdb",
      "Username": "labuser",
      "SSLMode": "prefer"
    }
  }
}
EOF
ras1putra@ras1putra:~/docker-lab/bab-5$
```

Analisis: Menyiapkan skrip DDL 01-schema.sql untuk membuat tabel students beserta index dan data awal, serta menyusun servers.json untuk auto-registrasi koneksi pgAdmin

Pembuatan skrip otomatisasi backup database (backup.sh)

```
ras1putra@ras1putra:~/docker-lab/bab-5$ cat << 'EOF' > scripts/backup.sh
#!/usr/bin/env bash
set -Eeuo pipefail

cd "$(dirname "$0")/.."

if [[ ! -f .env ]]; then
    echo "[FAIL] File .env tidak ditemukan." >&2
    exit 1
fi

set -a
source .env
set +a

mkdir -p backup

timestamp="$(date -u +%Y%m%d_%H%M%SZ)"
dump_file="backup/${POSTGRES_DB}-${timestamp}.dump"
checksum_file="${dump_file}.sha256"

docker compose exec -T postgres-db \
    pg_dump \
    --username "$POSTGRES_USER" \
    --dbname "$POSTGRES_DB" \
    --format custom \
    --no-owner \
    --no-privileges \
    > "$dump_file"

test -s "$dump_file"
sha256sum "$dump_file" > "$checksum_file"

echo "[PASS] Backup: $dump_file"
echo "[PASS] Checksum: $checksum_file"
EOF
```

Analisis: Menyusun skrip backup.sh untuk mengekstrak arsip binary custom (pg_dump -Fc), menonaktifkan ownership untuk portabilitas, dan menghasilkan nilai hash SHA256

Pembuatan skrip pengujian pemulihan database (restore-test.sh)

```
ras1putra@ras1putra:~/docker-lab/bab-5$ cat << 'EOF' > scripts/restore-test.sh
#!/usr/bin/env bash
set -Eeuo pipefail

cd "$(dirname "$0")/.."

if [[ ! -f .env ]]; then
    echo "[FAIL] File .env tidak ditemukan." >&2
    exit 1
fi

set -a
source .env
set +a

dump_file="${1:-}"
restore_database="${POSTGRES_DB}_restore_test"

if [[ -z "$dump_file" || ! -s "$dump_file" ]]; then
    echo "Penggunaan: $0 backup/nama-file.dump" >&2
    exit 1
fi

if [[ -f "${dump_file}.sha256" ]]; then
    sha256sum --check "${dump_file}.sha256"
fi

docker compose exec -T postgres-db \
    dropdb --username "$POSTGRES_USER" --if-exists "$restore_database"

docker compose exec -T postgres-db \
    createdb --username "$POSTGRES_USER" "$restore_database"

docker compose exec -T postgres-db \
    pg_restore \
    --username "$POSTGRES_USER" \
    --dbname "$restore_database" \
    --no-owner \
    --no-privileges \
    < "$dump_file"

docker compose exec -T postgres-db \
    psql \
    --username "$POSTGRES_USER" \
    --dbname "$restore_database" \
    --command "SELECT COUNT(*) AS total_students FROM students;"

echo "[PASS] Restore berhasil ke database: $restore_database"
EOF
ras1putra@ras1putra:~/docker-lab/bab-5$
```

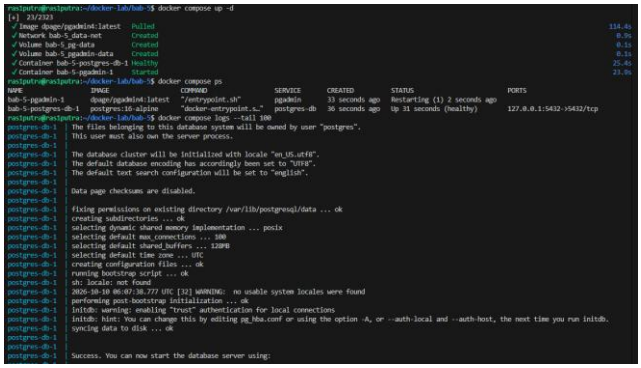
Analisis: Menyusun skrip pengujian restorasi otomatis yang memvalidasi checksum dan memulihkan data ke database uji terpisah (labdb_restore_test) tanpa merusak database utama

Validasi izin eksekusi berkas dan layanan compose

```
ras1putra@ras1putra:~/docker-lab/bab-5$ chmod +x scripts/backup.sh scripts/restore-test.sh
ras1putra@ras1putra:~/docker-lab/bab-5$
find . -maxdepth 3 -type f | sort
./compose.yaml
./.env
./.env.example
./.gitignore
./init/01-schema.sql
./pgadmin/servers.json
./scripts/backup.sh
./scripts/restore-test.sh
ras1putra@ras1putra:~/docker-lab/bab-5$ docker compose config --services
postgres-db
pgadmin
ras1putra@ras1putra:~/docker-lab/bab-5$
```

Analisis: Memberikan izin eksekusi (chmod +x) pada skrip shell, memvalidasi kelengkapan pohon direktori proyek, dan memastikan kedua service terdaftar valid pada Compose

Deployment multi-container dan inspeksi status healthcheck



Analisis: Menjalankan stack multi-container secara detached; service postgres-db berhasil menyelesaikan bootstrap cluster dan mencapai status Up (healthy)

Verifikasi inisialisasi skema tabel dan data awal via psql

```
ras1putra@ras1putra:~/docker-lab/bab-5$ docker compose exec -T postgres-db \
  psql -U labuser -d labdb \
  -c "\dt"
         List of relations
 Schema |   Name   | Type  |  Owner
--------+----------+-------+---------
 public | students | table | labuser
(1 row)

ras1putra@ras1putra:~/docker-lab/bab-5$ docker compose exec -T postgres-db \
  psql -U labuser -d labdb \
  -c "SELECT id, nrp, name, created_at FROM students ORDER BY id;"
 id |   nrp    |      name      |          created_at
----+----------+----------------+-------------------------------
  1 | 31230001 | Mahasiswa Satu | 2026-10-10 06:07:47.391125+00
  2 | 31230002 | Mahasiswa Dua  | 2026-10-10 06:07:47.391125+00
(2 rows)

ras1putra@ras1putra:~/docker-lab/bab-5$
```

Analisis: Memvalidasi bahwa tabel students otomatis dibuat oleh init script saat volume pertama kali kosong, dan membaca 2 record awal data mahasiswa via psql

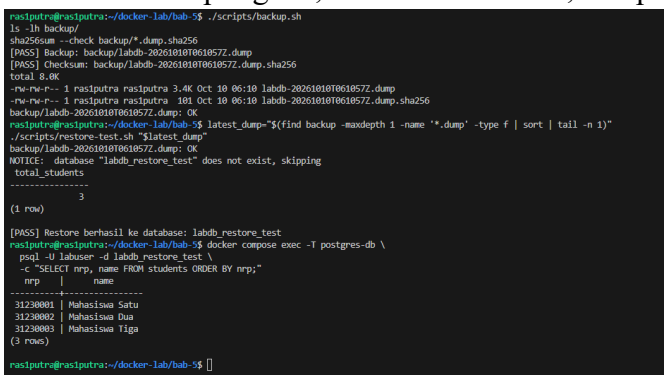
Pengujian ketahanan persistensi data pada named volume

```
ras1putra@ras1putra:~/docker-lab/bab-5$ docker compose exec -T postgres-db \
  psql -U labuser -d labdb \
  -c "INSERT INTO students(nrp, name) VALUES ('31230003', 'Mahasiswa Tiga') ON CONFLICT DO NOTHING;"
INSERT 0 1
ras1putra@ras1putra:~/docker-lab/bab-5$ docker compose down
[+] down 3/3
 ✓ Container bab-5-pgadmin-1     Removed
 ✓ Container bab-5-postgres-db-1 Removed
 ✓ Network bab-5_data-net        Removed
ras1putra@ras1putra:~/docker-lab/bab-5$ docker compose up -d
[+] up 3/3
 ✓ Network bab-5_data-net        Created
 ✓ Container bab-5-postgres-db-1 Healthy
 ✓ Container bab-5-pgadmin-1     Started
ras1putra@ras1putra:~/docker-lab/bab-5$ docker compose exec -T postgres-db \
  psql -U labuser -d labdb \
  -c "SELECT nrp, name FROM students ORDER BY nrp;"
   nrp    |      name
----------+----------------
 31230001 | Mahasiswa Satu
 31230002 | Mahasiswa Dua
 31230003 | Mahasiswa Tiga
(3 rows)

ras1putra@ras1putra:~/docker-lab/bab-5$
```

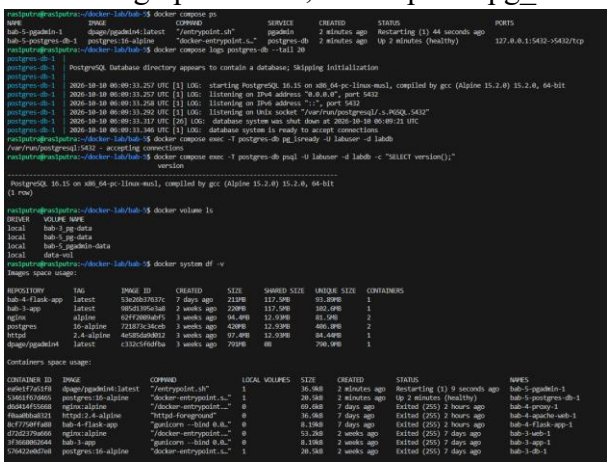
Analisis: Memasukkan data ke-3 (Mahasiswa Tiga) lalu menghancurkan container via docker compose down; setelah dijalankan ulang, data tetap utuh tersimpan di named volume pg-data

Eksekusi backup logical, validasi checksum, dan pengujian restore




Analisis: Eksekusi backup.sh menghasilkan berkas dump 3.4 KB berstatus SHA256 OK, dan restore-test.sh sukses memulihkan skema beserta 3 record mahasiswa ke database labdb_restore_test

Monitoring operasional, health probe pg_isready, dan resource docker




Analisis: Probe pg_isready mengonfirmasi database menerima koneksi socket, engine berjalan di versi PostgreSQL 16.15, dan volume bab-5_pg-data aktif mengamankan data stateful

## Verifikasi dan Skenario Pengujian

- PostgreSQL berjalan dengan volume pg-data: Direktori /var/lib/postgresql/data terikat persisten ke volume bab-5_pg-data.
- Tabel students dibuat otomatis dari init script: Skrip 01-schema.sql sukses dieksekusi otomatis oleh entrypoint saat bootstrap kluster.
- pgAdmin dapat login dan terkoneksi ke database: Terhubung via bridge network data-net ke host internal postgres-db:5432.
- Backup menghasilkan file dump di direktori host: Skrip backup.sh menghasilkan file labdb-*.dump (3.4 KB) dan checksum SHA256 valid.
- Restore sudah diuji nyata: Skrip restore-test.sh sukses memulihkan seluruh skema dan 3 data mahasiswa ke database uji labdb_restore_test.

## Evaluasi dan Latihan Mandiri

1. Mengapa init script tidak dijalankan ulang saat volume lama ada?
   Karena PostgreSQL hanya menjalankan init script saat pertama kali membuat database dan direktori datanya masih kosong. Jika volume lama sudah berisi data, script tersebut akan dilewati agar data yang sudah ada tidak terganggu

2. Risiko menaruh password database di compose.yaml?
   Password bisa ikut terekspos jika file diunggah ke repositori publik. Selain itu, informasi konfigurasi container juga bisa dilihat melalui docker inspect. Untuk mengurangi risiko, password bisa disimpan di file .env dengan akses terbatas atau menggunakan Docker Secrets
3. Cara membuktikan backup dapat dipulihkan?
   Caranya dengan mencoba memulihkan backup ke database terpisah, misalnya labdb_restore_test. Setelah itu, checksum SHA256 bisa diperiksa dan jumlah baris data dibandingkan untuk memastikan hasil pemulihan sesuai dengan data asli
4. Perbedaan logical backup pg_dump vs raw volume mentah?
   pg_dump menyimpan struktur dan isi database dalam format logis sehingga lebih mudah dipindahkan ke lingkungan lain. Sementara itu, raw volume backup menyalin file database secara langsung. Cara kedua perlu lebih hati-hati karena menyalin file saat database masih aktif dapat menghasilkan backup yang tidak konsisten jika dilakukan tanpa prosedur yang tepat
5. Dampak docker compose down -v terhadap database?
   Menghapus seluruh container, network, sekaligus named volume pg-data secara permanen, mengakibatkan hilangnya seluruh data jika belum ada backup dump

**Catatan Penggunaan AI dalam Praktikum**

Dalam praktikum ini, Generative AI (Gemini) digunakan sebagai teman diskusi untuk membantu penulisan analisis, merapikan kalimat agar lebih ringkas dan mudah dipahami, serta mencari solusi saat menghadapi kendala teknis selama praktikum.